

Interview Questions — Java - Hibernate Framework

Total Questions: 39

1. What is ORM (Object Relational Mapping) and why is it used in Java applications?

Hint: Think about bridging the gap between object-oriented code and relational databases.

2. Why would you choose Hibernate over using plain JDBC for database operations?

Hint: Consider code complexity, portability, and features like automatic mapping.

3. Explain the purpose of the SessionFactory and Session objects in Hibernate.

Hint: One manages resources for the whole application, the other is for individual database operations.

4. How do you configure Hibernate using the hibernate.cfg.xml file?

Hint: Mention database connection details and mapping resources.

5. What is the difference between configuring Hibernate via XML and using annotations?
-

6. In Hibernate, how do you map a Java class to a database table using annotations?

Hint: Think about which annotations are necessary on the class and its fields.

7. What does the @Entity annotation do in a Hibernate entity class?
-

8. What is the purpose of the @Id and @GeneratedValue annotations in Hibernate?
-

9. How would you map a field in a Java class to a specific column name in a database table using Hibernate?

Hint: Focus on the annotation that lets you set column properties.

10. Explain the difference between the transient and persistent states of an object in Hibernate.

Hint: Consider when an object is associated with a Hibernate session and when it isn't.

11. Suppose you have an Event entity for a campus portal. Which annotations would you use to ensure it is properly mapped to the EVENTS table?
-

12. What is a one-to-many relationship in Hibernate, and how do you represent it using annotations?

Hint: Think about which annotation is placed on the collection field.

13. How does the @JoinColumn annotation work in Hibernate relationship mapping?

14. Explain the difference between @OneToMany and @ManyToOne mappings in Hibernate.

15. What is cascading in Hibernate relationships, and why is it useful?

Hint: Consider what happens to child entities when you perform actions on the parent.

16. What are fetch types in Hibernate? Briefly explain the difference between LAZY and EAGER fetch.

17. Describe how you would map a scenario where one user can host multiple events using Hibernate annotations.

18. Write a simple Java entity class for a 'User' with an auto-generated primary key and a name field, using Hibernate annotations.

19. How would you retrieve all events hosted by a specific user using Hibernate Query Language (HQL)?

Hint: Think about writing a query that joins or filters based on a user property.

20. What are the basic CRUD operations in Hibernate, and which methods would you use for each?

21. Write a Hibernate code snippet to update the name of an Event entity given its ID.

22. How can you implement pagination when retrieving a large list of records using Hibernate?

Hint: Look for methods that set the starting point and maximum results.

23. What is the Criteria API in Hibernate and when would you use it instead of HQL?

24. Suppose you need to delete all events for a user who is leaving the portal. How would you do this efficiently in Hibernate?

25. How do you integrate Hibernate with Spring Boot in a Java application?

Hint: Think about dependencies, configuration, and automatic setup.

26. What is the difference between Hibernate's first-level cache and second-level cache?

Hint: Consider the scope of each cache and how long they persist data.

27. Name two popular second-level cache providers for Hibernate and briefly describe their purpose.

28. How can enabling caching improve the performance of a Hibernate application?

29. What are some common performance tuning tips when working with Hibernate?

30. You notice that your Hibernate queries are running slowly even after enabling second-level cache. What steps would you take to troubleshoot and improve performance?

Hint: Think about query structure, cache configuration, and database indexing.

31. In your mini project, how did you ensure that both Users and Events entities were correctly annotated and persisted using Hibernate?

32. Describe how you implemented the one-to-many relationship between Users and Events in your Event Manager project.

33. Suppose you want to display all events for a user in your Event Manager application with pagination. How would you implement this using Hibernate?

34. If you had previously built your project using JDBC, what changes did you make to migrate to Hibernate for persistence?

35. How would you use an AI tool like ChatGPT or GitHub Copilot to help you write Hibernate entity classes or queries? What would you check before using the code in your project?

Hint: Think about code generation, syntax suggestions, and the need to verify correctness and security.

36. How could AI tools assist you in troubleshooting a Hibernate mapping or configuration error, and why is it important to understand the underlying concepts yourself?

37. Integrative: Explain how annotations like @Entity, @Table, and @OneToMany work together to map complex relationships between Java classes and database tables in Hibernate.

38. Integrative: Describe how Hibernate's caching mechanisms can impact the performance of CRUD operations, especially when dealing with relationships like one-to-many.

- 39.** Integrative: In your mini project, if you had to add a new feature that allows users to search for events by date range, how would you design the entity, query, and performance considerations using Hibernate?
-

Generated by TOPS Technologies Career Center — for interview preparation.